



Labirentten Kaçış

Assembly 8086 ile Doğrudan Donanım Programlama

Bilgisayar mühendisliğinin temellerini anlama: düşük seviyeli programlama ile gerçek zamanlı oyun geliştirme

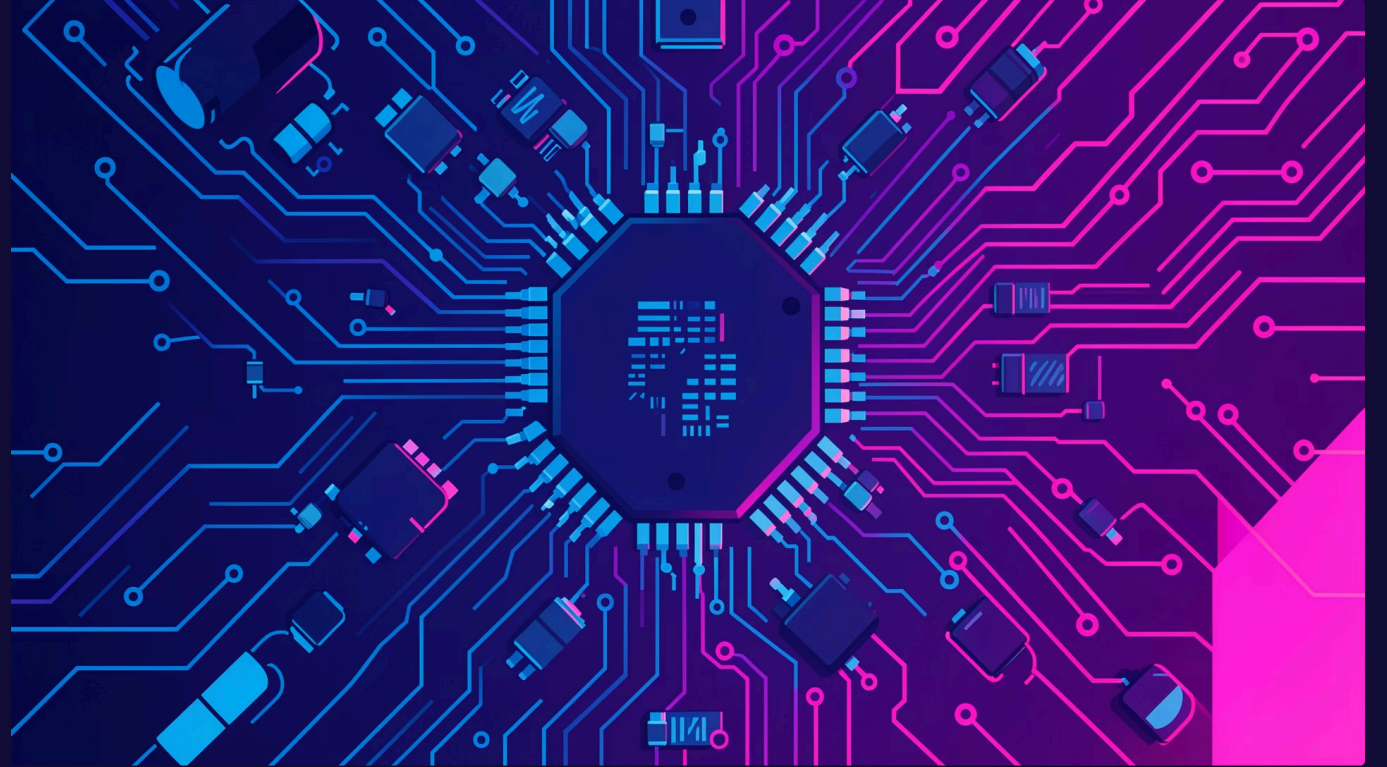
Projenin Amacı

Eğitimsel Değer

Assembly dilinin donanıma yakınlığı sayesinde bellek yönetimi, kesme mekanizmaları ve sistem kaynaklarına doğrudan erişim gibi temel kavramları pratik olarak öğrenmek.

Teknik Hedefler

- VGA belleğine doğrudan yazma
- Gerçek zamanlı klavye giriş işlemi
- Donanım saatinden zamanlama
- Bellek optimizasyonu teknikleri



Oyun Mekanığı ve Kuralları



3 Harita

20x20 boyutlarında farklı
ASCII tabanlı labirent yapıları



180 Saniye

Her harita için gerçek
zamanlı süre sınırı (3 dakika)



Çarpışma Algılama

Duvarlardan geçememe ve hedef (X) noktasına ulaşma

Teknik Altyapı

1

emu8086 Simülatörü

8086 mikroişlemcisi emülasyonu ve Assembly derleyici. MS-DOS ortamını simüle ederek gerçek donanım davranışını taklit eder.

2

Video Modu 03h

Standart 80x25 karakterlik text mode. Her karakter 8x16 piksel boyutunda, 16 renk destekli VGA uyumlu mod.

3

Doğrudan Donanım Erişimi

İşletim sistemi soyutlamalarını aşarak VGA RAM'e (B800h segment) doğrudan yazma. Maksimum performans için kesmeleri minimize etme.

VGA Belleğine Doğrudan Erişim

Bellek Organizasyonu

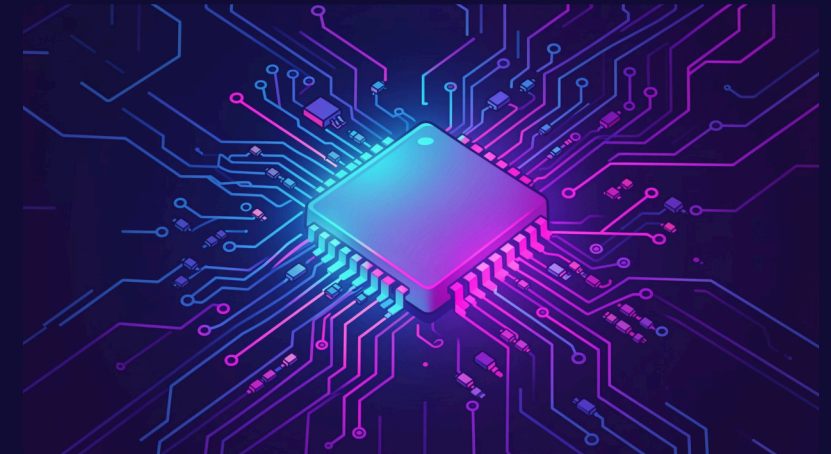
Ekran kartının **B800h** segment adresine ES yazmacı ile erişilir. Her karakter 2 byte ile temsil edilir:

- Byte 1: ASCII karakter kodu
- Byte 2: Renk ve özellik bilgisi

80 sütun x 25 satır = 2000 karakter kapasitesi

Neden Kesme Kullanılmadı?

DOS yazdırma kesmeleri (INT 21h) ekranı titretir ve performans kaybına neden olur. Doğrudan bellek yazımı anlık güncelleme sağlar.



Sistem Kesmeleri (Interrupts)



INT 16h – Klavye Girişi

Asenkron (non-blocking) tuş okuma. Oyuncu tuşa basmasa bile oyun döngüsü çalışmaya devam eder.



INT 21h AH=2Ch – Zaman

Bilgisayarın gerçek zaman saatinden (RTC) saniye okuma. 180 saniyelik sayacın oluşturulması.



INT 10h – Video Kontrol

Ekranı temizleme, video modu ayarlama ve imleç kontrolü. Sistem düzeyinde görsel operasyonlar.

Bellek Optimizasyonu ve Harita Yönetimi

Harita Yapısı

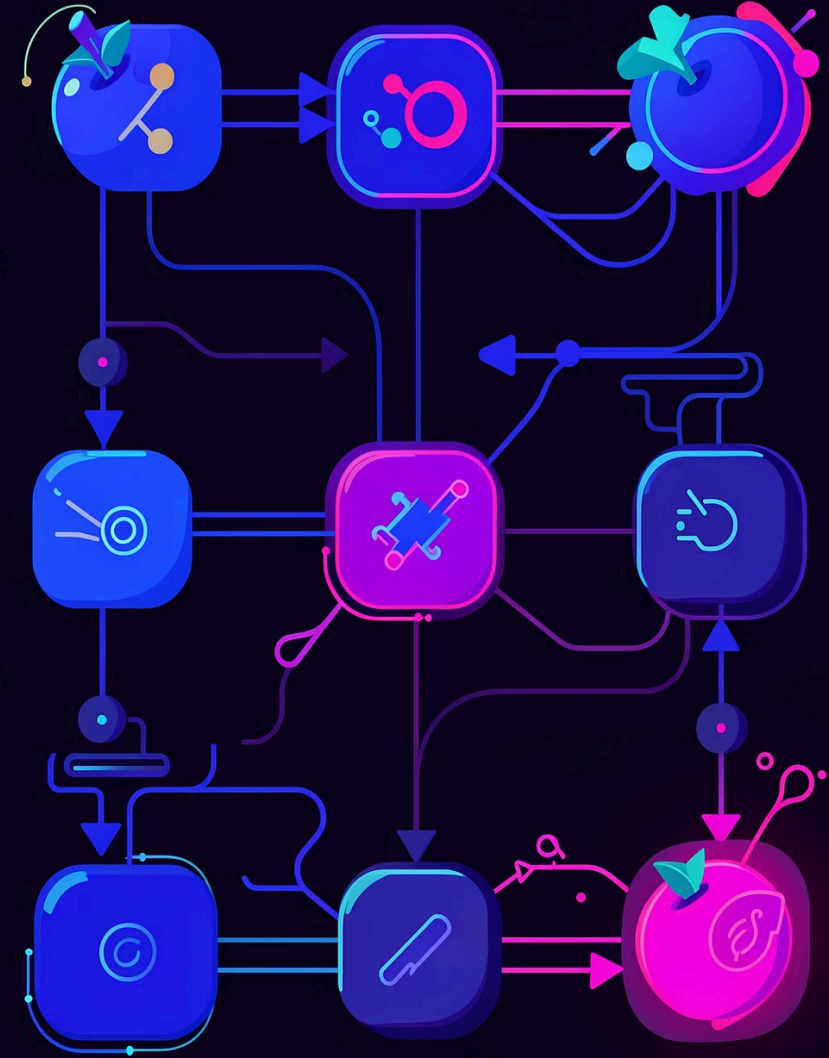
Her harita 400 elemanlı tek boyutlu DB dizisi olarak tanımlanır. Matris indeksleme için $Y * 20 + X$ formülü kullanılır.

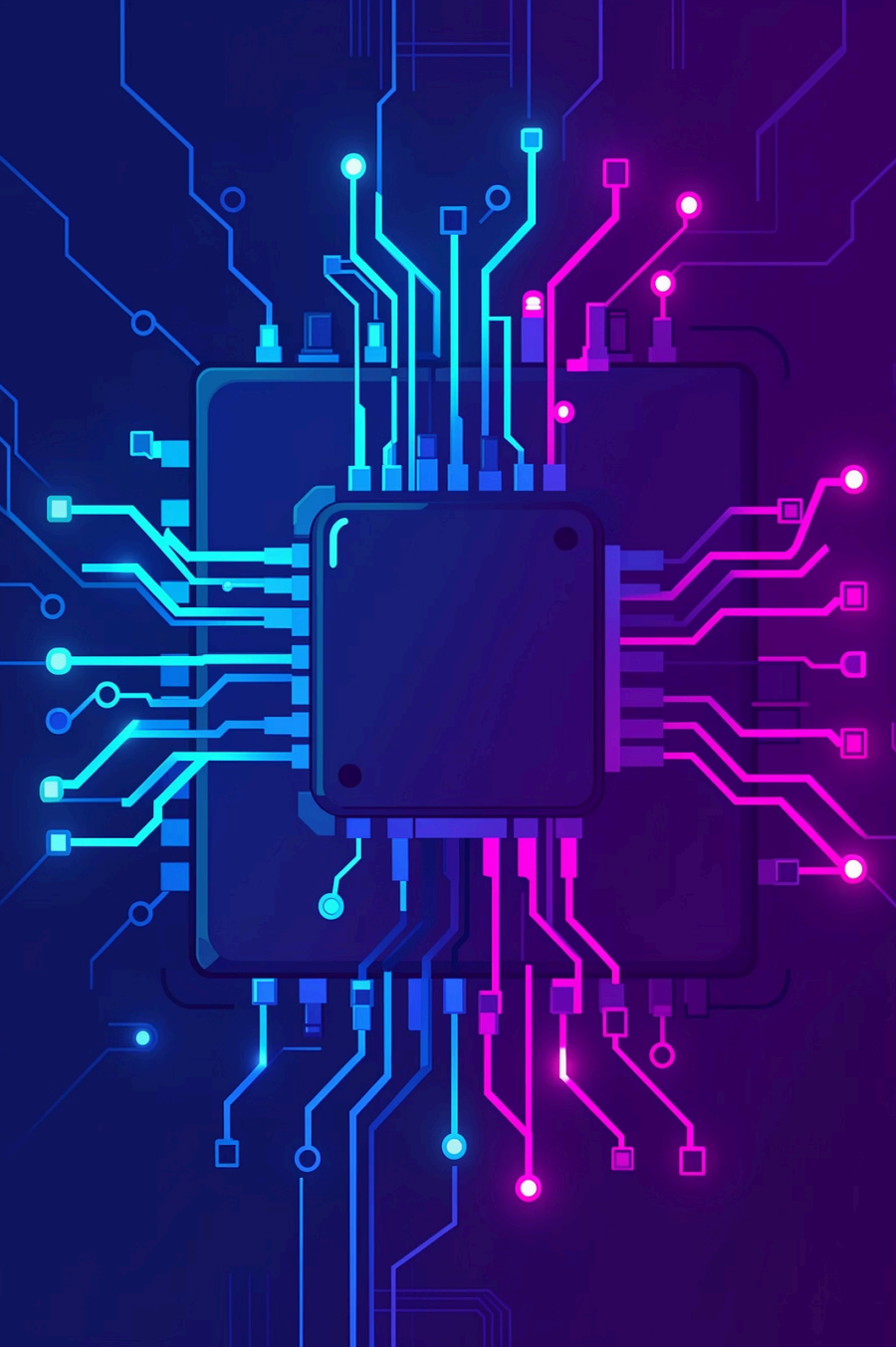
Dinamik Yükleme

REP MOVSB komutu ile donanım hızında harita kopyalama. 400 byte'lık harita verisi CPU tarafından doğrudan taşınır.

Çarpışma Algoritması

- Adım 1:** Hedef indeksi hesapla
- Adım 2:** CURRENT_MAP'den değeri oku
- Adım 3:** 1 ise duvar (çarpışma), 2 ise hedef (kazanma)





Karşılaşılan Zorluklar

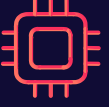
Problem: MUL Komutunun Yan Etkisi

Assembly'de çarpma işlemi (MUL) donanımsal olarak DX yazmacını ezmesi nedeniyle koordinat kaybı yaşanır.

Çözüm: Stack Koruması

Kritik yazmaçların PUSH ile stack'e kaydedilmesi ve MUL sonrası POP ile geri çağırılması ile veri bütünlüğü sağlanır.

Öğrenilen Kavramlar



Yazmaç Yönetimi

AX, BX, CX, DX gibi genel amaçlı yazmaçların rol ve kullanım alanları. Segment yazmaçları (DS, ES) ile bellek segmentasyonu.



Kesme Mekanizmaları

Donanım ve yazılım kesmeleri arasındaki fark. BIOS/DOS servisleri çağırma ve sistem kaynaklarına erişim.



Bellek Hiyerarşisi

Stack, data segment ve video RAM arasındaki farklar. Pointer kullanarak bellek adresleme teknikleri.



Performans Optimizasyonu

Kesmeler yerine doğrudan bellek erişimi. CPU komut seti ve pipeline optimizasyonu temelleri.

Projeden Çıkarılan Sonuçlar

Teknik Başarılar

- VGA RAM'e doğrudan erişim
- Gerçek zamanlı oyun döngüsü
- 3 harita ile dinamik içerik
- Çarpışma algılama sistemi

Eğitimsel Katkı

Düşük seviyeli programlama ile donanım arasındaki ilişkiyi anlama. Bellek yönetimi, kesme mekanizmaları ve sistem kaynaklarına doğrudan erişim becerileri kazanma.

